AI EXAMGUARD · THESIS DOCUMENTATION

Model Training Methodology

Datasets, preprocessing, training procedure, evaluation protocol and tooling for the machine-learning components of the AI ExamGuard online proctoring system.

Source commit 537b10e **Compiled** 4 September 2026 **Trained components** 3 **Pre-trained, unmodified** 3

1. Scope and overall approach

AI ExamGuard does not use a single model. Proctoring is performed by four independent detectors whose discrete outputs are combined by a fitted statistical model. Of the six algorithmic components, three were trained or fine-tuned for this system and three are used pre-trained without modification.

COMPONENT	ALGORITHM	STATUS	TRAINING DATA
Phone detection	YOLOv8s	**Fine-tuned**	Kaggle + MSU OEP + own captures
Risk scoring	Logistic regression	**Trained**	MSU OEP ground truth (24 subjects)
Face recognition	LBPH	**Trained per student**	5 enrolment captures, at runtime
Face detection	YuNet (ONNX)	Pre-trained	—
Person counting	YOLOv8s (COCO)	Pre-trained	—
Pose / wrist keypoints	YOLOv8n-pose	Pre-trained	—

Design constraint carried throughout. Every trained component was chosen to remain interpretable. Logistic regression was selected over a higher-capacity model specifically so the learned coefficients still read as "how much does each signal move the risk score" — preserving the transparency of the rule-based formula it replaced, while deriving the weights from data rather than assumption.

2. Datasets

2.1 MSU Online Exam Proctoring (OEP) dataset

The primary real-world source. Provides webcam recordings of subjects performing scripted cheating behaviours, with `gt.txt` ground-truth annotations defining five cheat-event types and their time ranges. Used for both phone-detector training data and risk-model labels.

- **24 subjects** with usable ground truth (expanded from an initial 11).
- Four subjects (1, 6, 9, 24) contain confirmed phone-positive events, verified exhaustively against `gt.txt`.
- Frames extracted by `extract_oep_frames.py`; phone-relevant windows isolated by `extract_oep_phone_windows.py`.

2.2 Kaggle cell-phone detection dataset

Public single-class phone dataset (`samuelayman/cell-phone`), retrieved via the Kaggle API in `download_dataset.py`. Supplies general phone appearance variety that the OEP footage alone does not cover.

2.3 Purpose-collected live captures

Additional batches recorded specifically to cover a failure mode absent from both public sources: phones held *back-first*, where no screen is visible. COCO's "cell phone" class skews heavily toward front-facing, screen-visible phones, so this gap was material.

3. Preprocessing and annotation pipeline

3.1 Phone-detector data preparation

1. **Acquisition** — `download_dataset.py` (Kaggle API), `capture_live_frames.py` (own captures), `extract_oep_frames.py` (video → frames).
2. **Format normalisation** — `prepare_dataset.py` converts to YOLO layout. Class IDs are *remapped* to contiguous 0-indexed values rather than copied verbatim, because the raw labels use a non-zero class id that YOLO would reject.
3. **Semi-automatic annotation** — `auto_annotate_oep.py` pre-labels frames using existing model inference, clustering overlapping candidate boxes and keeping the highest-confidence box per cluster.
4. **Human review** — contact sheets (`build_phone_box_contact_sheets.py` , `_build_labeled_contact_sheet.py`) render candidate boxes as reviewable grids; `annotate_phone_backface_live_review*.py` drive manual verification.
5. **Label correction** — `apply_phone_label_corrections.py` and `fix_phone_face_labels.py` apply reviewed fixes back to the label files.
6. **Splitting** — `prepare_oep_split.py` produces train/val with subjects 09 and 11 reserved as validation; `make_frozen_holdout.py` carves a separate frozen test set (§5.1).

Annotation quality is not assumed. A fairness/labelling audit (`fairness_audit.py`) identified a device consistently mislabelled across all 11 subjects then in the set — 579 boxes — which were corrected. A separate correction batch was retrained twice and *regressed* the frozen holdout on both attempts, so it was dropped from the training set and production was left untouched. Negative results were retained rather than discarded.

3.2 Risk-model data preparation

1. **Poll extraction** — `extract_risk_polls.py` replays recordings through the real detection pipeline, caching per-poll detection results.
2. **Windowing** — `build_risk_windows.py` slides a trailing **120-second** window (matching the production `WINDOW_SECONDS`) to produce counts of face-lost, phone-detected and multiple-people events.
3. **Labelling** — each window is labelled against `gt.txt` cheat events.

Deliberate label exclusion. Cheat types that a webcam structurally cannot observe — for example internet use — are excluded rather than forced into a class. A window whose only overlapping cheat event is invisible to the camera is dropped from training entirely. Labelling it positive would train the model against noise it cannot see; labelling it negative would teach it that observed-cheating frames are clean. An earlier version marked any overlapping code as positive, which measurably harmed held-out performance.

3.3 Face-recognition preprocessing (runtime, per student)

Executed during enrolment rather than offline. Each of five captures is converted to greyscale, the largest face located by YuNet, cropped, and resized to a fixed **200 × 200**. A minimum sample count is enforced; enrolment is rejected with a corrective message if too few clear faces are found.

4. Training procedure

4.1 Phone-detection specialist (YOLOv8)

Fine-tuned from the existing COCO `yolov8s.pt` weights via the Ultralytics API.

PARAMETER	VALUE	NOTE
Base weights	`yolov8s.pt`	COCO pre-trained
Epochs	`30`	Default; early stopping active
Batch size	`8`	CPU-constrained
Image size	`640`	Standard YOLO input
Patience	`8`	Early-stop on val plateau
Device	`cpu`	Reproducible on the deployment host
Output	`phone_specialist.pt`	Single class

Why a separate model rather than fine-tuning in place. The dataset labels only "phone". Fine-tuning `yolov8s.pt` directly on a single-class dataset rebuilds YOLO's detection head to emit that one class, which would silently destroy the `person` class the multiple-people check depends on. The specialist therefore runs as a second inference pass alongside the untouched base model.

4.2 Risk-scoring model (logistic regression)

PARAMETER	VALUE
Estimator	`sklearn.linear_model.LogisticRegression`
Regularisation	`C = 1.0`
Class weighting	`class_weight="balanced"`
Features	`face_lost_count, phone_detected_count, multiple_people_count`
Window	`120 s trailing`
Production fit	`--final`
Artefact	`risk_model.joblib`

The production model is fitted on all 24 subjects rather than holding a permanent validation split, because leave-one-subject-out cross-validation (§5.2) supplies the generalisation estimate without needing a reserved set. The training script prints an in-sample fit check in this mode and explicitly labels it as *not* a held-out metric.

Output rescaling

OEP subjects were instructed to cheat, so the training positive rate (~65%) far exceeds a real exam population's. The fitted intercept encodes that inflated base rate: `predict_proba(0,0,0) ≈ 0.16`. Since a session with no detected violations must score zero, the probability is linearly rescaled against that zero-evidence floor. This recalibrates only the output scale and does not alter the learned relationships between signals.

$$\text{risk\_vision} = \max(0, (p - p_0) / (1 - p_0)) \times 100 \quad \text{where } p_0 = \text{predict\_proba}(0,0,0)$$

4.3 Face recognition (LBPH, per student)

Trained at enrolment time, not offline: `cv2.face.LBPHFaceRecognizer.train()` over the student's preprocessed captures, labelled with their student ID, serialised to `<student_id>.yml`. Only the derived recognition model is retained — the raw enrolment photographs are not stored.

5. Evaluation protocol

5.0 What is evaluated, and how

Each trained component is evaluated differently, because each answers a different question. The table states the evaluation design per component so no single metric is over-generalised.

COMPONENT	HELD-OUT UNIT	DESIGN	PRIMARY METRIC
Phone detector	Stratified frame slice	Frozen holdout, evaluated once per swap candidate	Precision, Recall, F1
Risk model	Whole subject	Leave-one-subject-out cross-validation	AUC with 95% CI
Face recognition	Genuine vs impostor pairs	Distance-distribution separation study	F1 at operating threshold
Corroboration rule	Full session sequence	Ordered replay of real recordings	Swept parameter comparison

Metric definitions

- **Precision** = TP / (TP + FP) — of the frames flagged as containing a phone, the proportion that genuinely did. Low precision means false accusations, which in a proctoring context is the more damaging error.
- **Recall** = TP / (TP + FN) — of the frames that genuinely contained a phone, the proportion detected. Low recall means missed cheating.
- **F1** = 2 · (P · R) / (P + R) — harmonic mean, used to select an operating threshold where precision and recall trade off against each other.
- **AUC** — area under the ROC curve, the probability that a randomly chosen cheating window is scored higher than a randomly chosen clean one. Threshold-independent, which is why it is used for the risk model, whose output is a continuous score rather than a decision.

> **Why precision is weighted more heavily than recall here.** A false positive accuses an innocent student and must be defensible on appeal; a false negative misses one instance of cheating in a system that also has other signals and a human reviewer. Thresholds were therefore not chosen to maximise recall — the corroboration rule (3 consecutive polls) exists specifically to trade recall away for precision.

Baseline comparison

The risk model is not evaluated in isolation. The training script re-evaluates the previous hand-set weighting (FACE_LOST 25, PHONE_DETECTED 30, MULTIPLE_PEOPLE 25, capped at 100, thresholded at 50) on the identical validation windows, so the gain attributable to training is measured against what it replaced rather than asserted.

5.1 Frozen holdout (phone detector)

A stratified random slice drawn proportionally from every training-eligible source — each OEP subject and each live-capture batch — with a fixed seed, then *physically moved* out of the annotation directory so it cannot be swept back into training.

Why it was necessary. The validation subjects (09, 11) had been reused across successive runs to decide whether each candidate model should replace production. Repeatedly using one set for a sequence of accept/reject decisions is test-set hill-climbing, not generalisation evidence. Worse, some of those comparisons also quoted accuracy on live-capture batches the candidate had just been trained on — a training-set number reported alongside a held-out one without distinction. The frozen holdout exists to break that loop, under a stated discipline: never train on it, never use it for threshold or checkpoint selection, evaluate once per genuine swap candidate, and report the number even when unflattering.

Metric definition

Two metrics are computed. The **presence** metric matches production exactly — deployment performs a pure presence check with no location awareness. An **IoU-gated localisation** metric is retained as a model-quality diagnostic. An earlier evaluation required IoU matching and omitted the hand-crop fallback pass, making it strictly weaker than the deployed pipeline and understating real recall.

5.2 Leave-one-subject-out cross-validation (risk model)

The model is refitted once per subject, each time holding out that entire subject. Whole subjects — never random window slices — are the held-out unit, because a window from a subject otherwise present in training tests almost nothing new.

METRIC	VALUE	BASIS
Pooled AUC	`0.813`	LOSO-CV, 24 subjects
95% confidence interval	`[0.712, 0.902]`	Tightened from [0.666, 0.941] at 11 subjects

The training script additionally evaluates the previous hand-set weights on the same windows, so any improvement from training is measured rather than assumed.

5.3 Threshold calibration studies

Operating thresholds were derived from measurement, each with a dedicated analysis script, not chosen by intuition.

THRESHOLD	VALUE	STUDY
LBPH match distance	`60.0`	`analyze_face_recognition_threshold.py` — F1-optimal (P 0.950, R 0.873)
Head-down pitch	`−35.0°`	`analyze_head_pose_thresholds.py`
Head-down duration	`25 s`	`analyze_head_down_duration.py`
Static-image difference	`8.0`	`analyze_static_image_threshold.py`
Phone corroboration	`3 of 3`	`evaluate_corroboration.py` — swept, sequence-ordered replay

5.4 Fairness sensitivity audit

A tier-1 audit (`fairness_audit.py`) compares detection performance across capture batches and conditions. Its stated limitation is recorded honestly: the dataset carries no documented skin-tone, gender, age or camera-quality labels, so the audit can establish that a performance gap exists between groups but cannot attribute its cause.

6. Tools and libraries

TOOL	VERSION	ROLE
Ultralytics YOLO	`8.4.104`	Fine-tuning and inference for detection and pose
PyTorch / TorchVision	`2.13.0 / 0.28.0`	YOLO backend; CPU build in deployment
OpenCV (contrib)	`5.0.0.93`	YuNet detection, LBPH recognition, solvePnP, image preprocessing
scikit-learn	`1.9.0`	Logistic regression, cross-validation, metrics
pandas	`3.0.5`	Feature frames for training and inference
NumPy	`2.5.1`	Array and matrix operations
joblib	`1.5.3`	Model serialisation
Matplotlib	`3.11.1`	Analysis plots and contact sheets
Kaggle API	–	Dataset retrieval

A dependency conflict worth documenting. Ultralytics transitively installs `opencv-python`, which silently overwrites `opencv-contrib-python`'s `cv2.face` module — removing LBPH while leaving `import cv2` working. The failure is therefore silent until face recognition is invoked. Both the container build and CI assert `hasattr(cv2.face, "LBPHFaceRecognizer_create")` so the build fails loudly instead of shipping a broken recogniser.

7. Reproducibility and limitations

- **Fixed seeds** are used for the frozen-holdout split so the partition is reproducible.
- **CPU-only inference** in deployment is deliberate, so results do not depend on GPU availability.
- **Negative results retained.** Two corrected-label retrainings and a second pose-based pitch signal were each tried, measured, found worse on held-out data, and reverted. They are documented rather than omitted.
- **Known unresolved gap.** LBPH compares texture and has no liveness concept; a printed photograph of the enrolled student scores within the genuine range. Frame-difference and blink-detection mitigations were both attempted and reached real ceilings. Closing this requires dense eyelid landmarks — a new model dependency, not threshold tuning.
- **Population validity.** OEP subjects were instructed to cheat, giving a positive rate far above any real cohort. This is corrected for in the score scaling but remains a caveat on the training distribution.
- **Re-evaluate before quoting.** The last recorded frozen-holdout figures (P 0.927, R 0.792, F1 0.854) predate subsequent model changes and should be regenerated and re-dated before being cited.

Compiled from the AI ExamGuard repository at commit 537b10e. Every parameter, threshold and version stated here was read from source, configuration or the installed dependency set rather than reproduced from documentation. Training and analysis scripts referenced by filename are located under `backend/training/`.